# INSTRUCTION FORMAT SPECIFICATION FOR A TWO-PASS ASSEMBLER

## Target Architecture: 32-bit x86 (IA-32)

### Group Technical Documentation

**Architecture:** IA-32 / 32-bit x86
**Instruction Prefixes:** Excluded
**Byte Ordering:** Little-Endian
**Assembler Type:** Two-Pass Assembler

---

# 1. Group Members and Responsibilities

| Person | Member | Assigned Responsibility |
|---|---|---|
| Person 1 | **Shruti** | Instruction Format & Opcode Encoding |
| Person 2 | **Komal** | Addressing & Operand Encoding |
| Person 3 | **Aditi** | Immediates, Jumps, Directives & Two-Pass Assembly |

The complete assembler specification is divided into three connected parts. Each member is responsible for one stage of the instruction-encoding process.

### Person 1 – Shruti

Shruti is responsible for determining **what operation the instruction represents and which opcode encoding should be selected**.

Her section covers:

- Introduction and scope
- IA-32 instruction format
- Overall instruction byte layout
- Primary 1-byte opcodes
- `0x0F` two-byte opcode format
- `d` direction bit
- `s` sign-extension bit
- Opcode lookup tables
- Register-encoded opcodes
- Opcode selection procedure

**Main question:**

> How does the assembler determine the opcode portion of an instruction?

---

### Person 2 – Komal

Komal is responsible for determining **how the operands and memory addresses are represented using ModR/M, SIB, registers, and displacement fields**.

Her section covers:

- 32-bit register encoding
- ModR/M byte
- MOD, REG, and R/M fields
- SIB byte
- SCALE, INDEX, and BASE fields
- Register addressing
- Memory addressing
- Displacement addressing
- Complex addressing expressions

Example:

```
MOV EAX, [EBX + ESI*4 + 0x10]
```

**Main question:**

> How does the assembler represent the operands and memory address?

---

### Person 3 – Aditi

Aditi is responsible for determining **how constants, labels, relative addresses, data directives, sections, and two-pass processing are handled**.

Her section covers:

- Immediate constants
- 8-bit and 32-bit immediates
- 8-bit and 32-bit displacements
- Little-Endian representation
- Relative `JMP`
- Relative `JNE`
- Relative address calculation
- `DB`, `DW`, and `DD`
- `SECTION .text`
- `SECTION .data`
- Symbol table
- Pass 1
- Pass 2
- Final machine-code generation

**Main question:**

How does the assembler resolve values, labels, sections, and generate the final bytes?

---

# 2. Introduction

The objective of this project is to design and document the **Instruction Format Specification for a Two-Pass Assembler targeting the 32-bit x86 (IA-32) architecture**.

The specification describes how assembly language instructions are converted into variable-length machine-code instructions.

Instruction prefixes are outside the scope of this specification.

IA-32 instructions can contain different fields depending on the instruction and its operands. The general instruction structure consists of an opcode followed by optional ModR/M, SIB, displacement, and immediate fields.

The complete instruction can therefore be represented as:

```
+----------+----------+----------+---------------+-----------+
|  OPCODE  |  ModR/M  |   SIB    | DISPLACEMENT  | IMMEDIATE |
+----------+----------+----------+---------------+-----------+
| 1/2 Byte | Optional | Optional |   0/1/4 B     | Optional  |
+----------+----------+----------+---------------+-----------+
```

Not every instruction contains every field.

---

# 3. Overall Assembler Encoding Process

The three members' responsibilities together form a complete instruction-encoding pipeline.

```
            Assembly Source Instruction
                      |
                      v
             +------------------+
             |   PERSON 1       |
             |     SHRUTI       |
             | Opcode Selection |
             +------------------+
                      |
                      v
             +------------------+
             |   PERSON 2       |
             |      KOMAL       |
             | Operand Encoding |
```

```
               | ModR/M + SIB     |
               +------------------+
                        |
                        v
               +------------------+
               |   PERSON 3       |
               |      ADITI       |
               | Immediates       |
               | Displacements    |
               | Labels/Jumps     |
               | Directives       |
               +------------------+
                        |
                        v
                Final Machine Code
```

The final machine code is constructed as:

```
Opcode
   +
ModR/M (if required)
   +
SIB (if required)
   +
Displacement (if required)
   +
Immediate (if required)
```

---

# 4. PART I – INSTRUCTION FORMAT & OPCODE ENCODING

## 4.1 Responsible Member: Shruti

The first stage of assembly is identifying the instruction operation and selecting its opcode.

IA-32 uses variable-length instructions. Excluding prefixes, an instruction can have one or two opcode bytes followed by optional fields.

---

## 4.2 Primary 1-Byte Opcode

Many IA-32 instructions use a single primary opcode byte.

Representative forms include:

| Instruction | Operand Form | Opcode |
|---|---|---|
| MOV | r/m32, r32 ; r32, r/m32 ; r32, imm32 | 89 /r ; 8B /r ; B8+rd id |
| ADD | r/m32, r32 ; r32, r/m32 | 01 /r ; 03 /r |
| SUB | r/m32, r32 ; r32, r/m32 | 29 /r ; 2B /r |
| CMP | r/m32, r32 ; r32, r/m32 | 39 /r ; 3B /r |
| JMP | rel8 ; rel32 | EB cb ; E9 cd |
| CALL | rel32 | E8 cd |
| AND | r/m32, r32 ; r32, r/m32 | 21 /r ; 23 /r |
| XOR | r/m32, r32 ; r32, r/m32 | 31 /r ; 33 /r |
| MUL | r/m32 | F7 /4 |
| OR | r/m32, r32 ; r32, r/m32 | 09 /r ; 0B /r |

Here, `/r` indicates that

---

## 4.3 Two-Byte Opcode

Some IA-32 instructions use `0x0F` followed by a second opcode byte.

For example:

```
JNE rel32
```

uses:

```
0F 85 cd cd cd cd
```

The structure is:

```
+------+--------------+-------------------+
| 0F   | Second Byte  | Optional Operands |
+------+--------------+-------------------+
```

---

## 4.4 Direction Bit

Some opcode formats contain a `d` direction bit.

It determines the direction of data transfer between the `REG` and `R/M` fields.

| d | Meaning |
|---|---|
| `0` | REG is source; R/M is destination |
| `1` | REG is destination; R/M is source |

For example:

```
MOV [EBX], EAX
```

uses:

```
89 /r
```

while:

```
MOV EAX, [EBX]
```

uses:

```
8B /r
```

---

## 4.5 Sign-Extension Bit

Some arithmetic encodings contain an `s` bit.

When:

```
s = 1
```

a smaller immediate is sign-extended to the required operand size.

For example:

```
05 → 00000005
FF → FFFFFFFF
```

This permits a smaller signed immediate to be used when it fits the required range.

---

## 4.6 Opcode Selection

The assembler cannot select an opcode using only the mnemonic.

It considers:

1. Mnemonic
2. Operand types
3. Operand size
4. Operand direction
5. Immediate/displacement requirements

The selection process is:

```
Assembly Instruction
        ↓
Identify Mnemonic
        ↓
Identify Operands
        ↓
Determine Operand Type and Size
        ↓
Determine Direction / Immediate Form
        ↓
Search Opcode Table
        ↓
Select Opcode
        ↓
Generate Additional Fields
```

This opcode-selection procedure is defined in the uploaded Person 1 specification.

---

# 5. PART II – ADDRESSING & OPERAND ENCODING

## 5.1 Responsible Member: Komal

After the opcode is selected, the assembler must represent the instruction's operands.

This is primarily achieved using:

- Register codes
- ModR/M
- SIB
- Displacement

---

## 5.2 32-bit Register Encoding

IA-32 general-purpose registers use 3-bit codes.

| Register | Binary | Decimal | Hex |
|---|---|---|---|
| EAX | `000` | 0 | 0 |
| ECX | `001` | 1 | 1 |
| EDX | `010` | 2 | 2 |
| EBX | `011` | 3 | 3 |
| ESP | `100` | 4 | 4 |
| EBP | `101` | 5 | 5 |
| ESI | `110` | 6 | 6 |
| EDI | `111` | 7 | 7 |

These codes are reused in the ModR/M and SIB fields.

---

# 6. ModR/M Byte

The ModR/M byte specifies register or memory operands.

Its bit layout is:

```
 7   6 | 5   4   3 | 2   1   0
+-------+-----------+-----------+
|  MOD  |    REG    |    R/M    |
```

```
+-------+-----------+-----------+
  2 bits    3 bits      3 bits
```

| Field | Size | Purpose |
|---|---|---|
| MOD | 2 bits | Selects addressing mode |
| REG | 3 bits | Selects register or opcode extension |
| R/M | 3 bits | Selects register or memory form |

---

## 6.1 MOD Field

| MOD | Meaning |
|---|---|
| `00` | Memory addressing, normally without displacement |
| `01` | Memory addressing with signed 8-bit displacement |
| `10` | Memory addressing with 32-bit displacement |
| `11` | Register addressing |

---

# 7. SIB Byte

The SIB byte provides scaled-index addressing.

Its format is:

```
 7   6 | 5   4   3 | 2   1   0
+-------+-----------+-----------+
| SCALE |   INDEX   |   BASE    |
+-------+-----------+-----------+
  2 bits    3 bits      3 bits
```

| Field | Size | Purpose |
|---|---|---|
| SCALE | 2 bits | Selects index multiplier |
| INDEX | 3 bits | Selects index register |
| BASE | 3 bits | Selects base register |

Scale encoding:

| SCALE | Multiplier |
|---|---|
| 00 | ×1 |
| 01 | ×2 |
| 10 | ×4 |
| 11 | ×8 |

---

# 8. Complex Addressing Example

Consider:

```
MOV EAX, [EBX + ESI*4 + 0x10]
```

The assembler identifies:

```
Destination = EAX
Base        = EBX
Index       = ESI
Scale       = 4
Displacement = 0x10
```

Since a SIB byte is required:

```
R/M = 100
```

Since the displacement fits in 8 bits:

```
MOD = 01
```

Destination EAX:

```
REG = 000
```

Therefore:

```
MODR/M = 01 000 100
       = 44
```

SIB:

```
SCALE = 10
INDEX = 110
BASE  = 011
```

Therefore:

```
SIB = 10 110 011
    = B3
```

The displacement is:

```
10
```

Final encoding:

```
8B 44 B3 10
```

This exact example is present in Komal's uploaded addressing specification.

---

# 9. PART III – IMMEDIATES, JUMPS, DIRECTIVES & TWO-PASS ASSEMBLY

## 9.1 Responsible Member: Aditi

After opcode and operand encoding are determined, the assembler must resolve constants, offsets, labels, and data.

This part handles values that may depend on addresses or symbols that are not known during the initial parsing stage.

---

# 10. Immediate Values

An immediate is a constant value stored directly in the instruction.

For example:

```
MOV EAX, 0x12345678
```

The `MOV r32, imm32` encoding uses:

```
B8 + rd
```

For EAX:

```
rd = 000
```

so:

```
Opcode = B8
```

The immediate is:

```
0x12345678
```

and is stored Little-Endian:

```
78 56 34 12
```

Final instruction:

```
B8 78 56 34 12
```

This encoding is also included in the group's existing instruction specification.

---

# 11. Immediate and Displacement Sizes

The assembler supports the following relevant sizes:

| Field | Size |
|---|---|
| `imm8` | 1 byte |
| `imm16` | 2 bytes |
| `imm32` | 4 bytes |
| `disp8` | 1 byte |
| `disp32` | 4 bytes |
| `rel8` | 1 byte |
| `rel32` | 4 bytes |

The general instruction specification permits immediate fields of 0, 1, 2, or 4 bytes and displacement fields of 0, 1, or 4 bytes.

---

# 12. Little-Endian Representation

IA-32 uses Little-Endian ordering.

For:

```
0x12345678
```

the byte representation is:

```
78 56 34 12
```

Similarly:

```
0x00000110
```

becomes:

```
10 01 00 00
```

Little-Endian ordering applies to:

- Multi-byte immediates
- Multi-byte displacements
- Relative offsets
- Data defined by `DW` and `DD`

---

# 13. Relative Addressing

Branch instructions such as:

```
JMP LABEL
JNE LABEL
```

use relative addressing.

The assembler calculates:

```
Relative Displacement =
Target Address − Address of Next Instruction
```

The address of the next instruction is used as the reference point rather than the beginning of the branch instruction.

---

# 14. JMP Encoding

Two forms are supported:

| Form | Opcode | Offset |
|---|---|---|
| `JMP rel8` | `EB cb` | 8-bit signed |
| `JMP rel32` | `E9 cd` | 32-bit signed |

For a short jump:

```
displacement = target - next_instruction
```

If the result fits the signed 8-bit range, the assembler can use:

```
EB cb
```

Otherwise, the 32-bit form can be selected:

```
E9 cd
```

---

# 15. JNE Encoding

Two forms are supported:

| Form | Opcode | Offset |
|---|---|---|
| `JNE rel8` | `75 cb` | 8-bit signed |
| `JNE rel32` | `0F 85 cd` | 32-bit signed |

The near `JNE` uses the two-byte opcode `0F 85`.

For example:

```
JNE address = 0x3000
Instruction size = 2
Target = 0x3020
```

Next instruction:

```
0x3002
```

Displacement:

```
0x3020 - 0x3002 = 0x1E
```

Final encoding:

```
75 1E
```

---

# 16. Data Directives

The assembler supports the following data directives:

| Directive | Meaning | Size |
|---|---|---|
| `DB` | Define Byte | 1 byte |
| `DW` | Define Word | 2 bytes |
| `DD` | Define Doubleword | 4 bytes |

### DB

```
DB 0x25
```

Output:

```
25
```

### DW

```
DW 0x1234
```

Output:

```
34 12
```

**DD**

```
DD 0x12345678
```

Output:

```
78 56 34 12
```

The `DW` and `DD` values are emitted in Little-Endian byte order.

---

# 17. SECTION .text

The `.text` section contains executable instructions.

Example:

```
SECTION .text

MOV EAX, 1
RET
```

The assembler generates machine-code bytes for the instructions in this section.

The `.text` location counter is advanced according to the size of each generated instruction.

---

# 18. SECTION .data

The `.data` section contains initialized data.

Example:

```
SECTION .data

VALUE DB 10
NUMBER DW 0x1234
COUNT DD 0x12345678
```

The generated bytes are:

```
VALUE:
10

NUMBER:
34 12

COUNT:
78 56 34 12
```

---

# 19. Labels and Symbol Table

Labels identify addresses in the program.

Example:

```
SECTION .text

START:
    MOV EAX, 1
    JMP END

END:
    RET
```

The assembler creates symbol-table entries for:

```
START
END
```

The symbol table conceptually contains:

| Symbol | Section | Address |
|---|---|---|
| START | `.text` | Address of MOV |
| END | `.text` | Address of RET |

The addresses are established during Pass 1.

---

# 20. Pass 1

Pass 1 is the **analysis and symbol-collection phase**.

Its main tasks are:

1. Read the source program.
2. Identify sections.
3. Identify labels.
4. Identify instructions.
5. Identify operands.
6. Identify data directives.
7. Determine instruction/data sizes.
8. Update location counters.
9. Build the symbol table.

Conceptually:

```
Source Program
      ↓
Parse Source
      ↓
Identify Sections
      ↓
Identify Labels
      ↓
Determine Sizes
      ↓
Update Location Counters
      ↓
Build Symbol Table
```

The existing group specification describes Pass 1 as recording labels/symbols, determining expected instruction lengths, and maintaining the location counter.

---

# 21. Pass 2

Pass 2 is the **final code-generation phase**.

Its main tasks are:

1. Read the source again.
2. Resolve symbols.
3. Calculate relative branch offsets.
4. Select final opcodes.
5. Generate ModR/M and SIB bytes.
6. Generate displacement fields.
7. Generate immediate fields.
8. Apply Little-Endian ordering.
9. Emit final machine-code/data bytes.

Conceptually:

```
Symbol Table
      ↓
Read Source Again
      ↓
Resolve Labels
      ↓
Calculate Relative Offsets
      ↓
Generate Opcode
      ↓
Generate ModR/M / SIB
      ↓
Generate Displacement
      ↓
Generate Immediate
      ↓
Little-Endian Conversion
      ↓
Final Output
```

---

# 22. Integration of All Three Members

The work of Shruti, Komal, and Aditi is sequentially connected.

Consider:

```
MOV EAX, [EBX + ESI*4 + 0x10]
```

### Step 1 – Shruti: Opcode Selection

Identify the instruction form:

```
MOV r32, r/m32
```

Select:

```
Opcode = 8B
```

---

### Step 2 – Komal: Operand Encoding

Identify:

```
Destination = EAX
Base = EBX
Index = ESI
Scale = 4
Displacement = 0x10
```

Generate:

```
ModR/M = 44
SIB = B3
disp8 = 10
```

---

### Step 3 – Aditi: Final Value/Byte Handling

The displacement is `0x10`, which fits in one byte.

Therefore:

```
disp8 = 10
```

No byte reversal is needed because it is one byte.

---

### Final Machine Code

```
8B 44 B3 10
```

This demonstrates how the three responsibilities combine to produce the final encoding. The same final encoding is documented in the Person 2 specification.

---

# 23. Complete Example – Immediate Instruction

Consider:

```
MOV EAX, 0x12345678
```

### Shruti

Instruction form:

```
MOV r32, imm32
```

Opcode:

```
B8 + rd
```

For EAX:

```
rd = 000
Opcode = B8
```

### Komal

No ModR/M or SIB is required because the destination register is encoded directly in the opcode.

### Aditi

Immediate:

```
0x12345678
```

Little-Endian:

```
78 56 34 12
```

### Final Machine Code

```
B8 78 56 34 12
```

---

# 24. Complete Example – Memory Addressing

Consider:

```
MOV EAX, [EBX]
```

### Shruti

Instruction form:

```
MOV r32, r/m32
```

Opcode:

```
8B /r
```

### Komal

```
MOD = 00
REG = EAX = 000
R/M = EBX = 011
```

Therefore:

```
ModR/M = 00 000 011
       = 03
```

### Aditi

No immediate or displacement is required.

### Final Machine Code

```
8B 03
```

This encoding is also present in the uploaded group specification.

---

# 25. Complete Example – Relative Branch

Consider:

```
JNE TARGET
```

### Shruti

Identify the instruction:

```
JNE rel8
```

or:

```
JNE rel32
```

Select the appropriate opcode:

```
75
```

for short form, or:

```
0F 85
```

for near form.

### Komal

No ModR/M or SIB is required.

### Aditi

Calculate:

```
displacement =
TARGET − address_of_next_instruction
```

Then encode the resulting displacement as `rel8` or `rel32`.

For a 32-bit displacement, apply Little-Endian ordering.

### Final Form

Short:

```
75 cb
```

Near:

```
0F 85 cd cd cd cd
```

---

# 26. Overall Division of Work

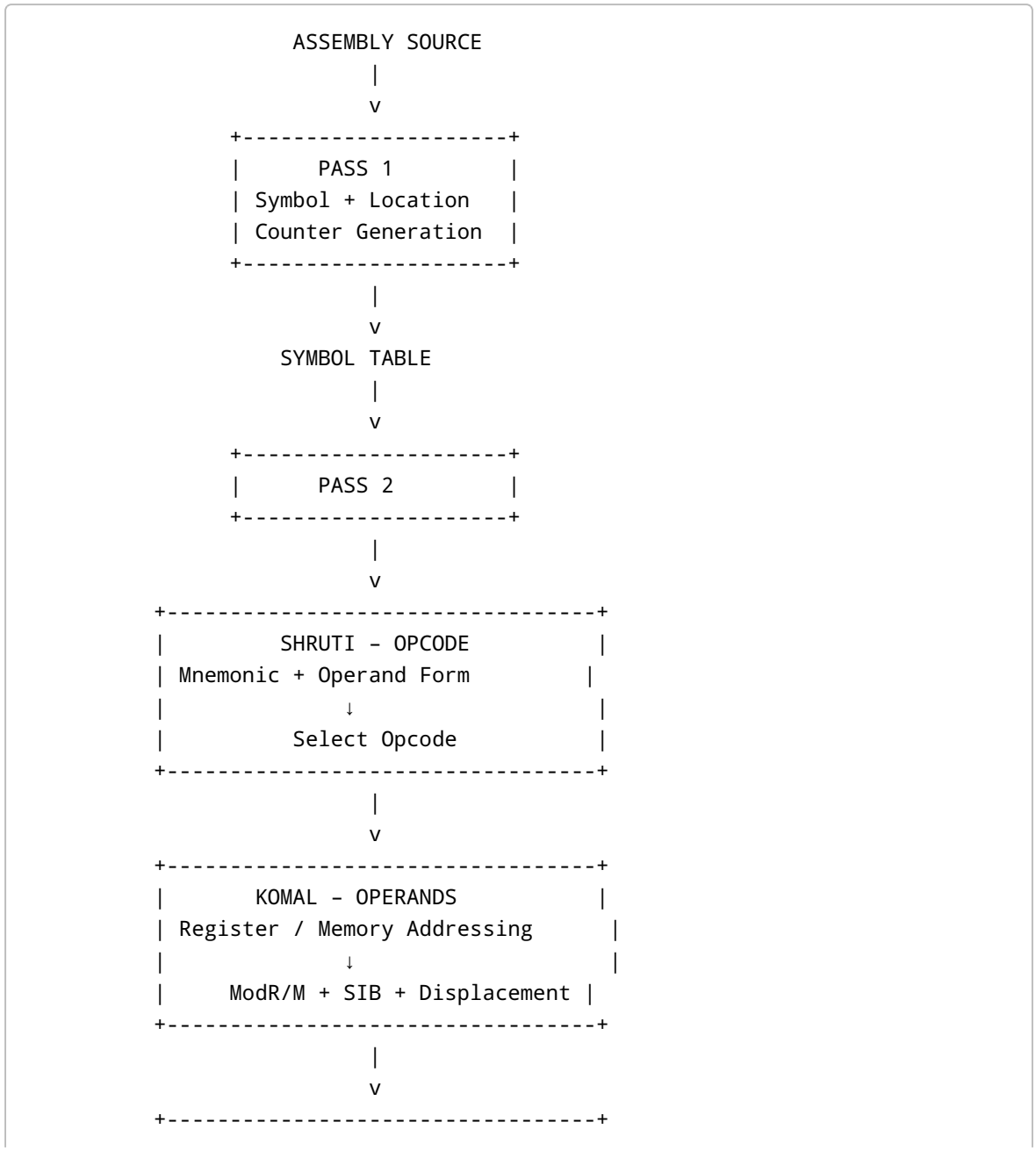
| Module | Shruti | Komal | Aditi |
|---|---|---|---|
| Introduction & Scope | ✓ | | |
| IA-32 Instruction Format | ✓ | | |
| Instruction Byte Diagram | ✓ | | |
| Primary Opcode | ✓ | | |
| `0F` Two-Byte Opcode | ✓ | | |
| `d` Direction Bit | ✓ | | |
| `s` Sign-Extension Bit | ✓ | | |
| Opcode Lookup Table | ✓ | | |
| Opcode Selection | ✓ | | |
| Register Encoding | | ✓ | |
| ModR/M | | ✓ | |
| MOD / REG / R/M | | ✓ | |
| SIB | | ✓ | |
| SCALE / INDEX / BASE | | ✓ | |
| Register Addressing | | ✓ | |
| Memory Addressing | | ✓ | |
| Displacement Addressing | | ✓ | ✓ |
| Complex Addressing | | ✓ | |
| Immediate Values | | | ✓ |
| 8/32-bit Immediate | | | ✓ |
| 8/32-bit Displacement | | ✓ | ✓ |
| Little-Endian | | | ✓ |
| Relative JMP | | | ✓ |
| Relative JNE | | | ✓ |
| Relative Offset Calculation | | | ✓ |
| DB / DW / DD | | | ✓ |
| `.text` Section | | | ✓ |
| `.data` Section | | | ✓ |

| Module | Shruti | Komal | Aditi |
|---|---|---|---|
| Labels / Symbol Table | | | ✓ |
| Pass 1 | | | ✓ |
| Pass 2 | | | ✓ |
| Final Code Generation | | | ✓ |

---

# 27. Final System Architecture

The complete specification can be summarized as follows:



```
          |        ADITI – VALUES            |
          | Immediate / Labels / Jumps       |
          | Directives / Relative Offsets    |
          |              ↓                   |
          | Little-Endian + Final Resolution |
          +----------------------------------+
                           |
                           v
                 FINAL MACHINE CODE
```

---

# 28. Conclusion

This group specification defines the major stages required to convert IA-32 assembly language into machine code using a two-pass assembler.

**Shruti** defines the **operation and opcode selection**, determining which opcode corresponds to the instruction form.

**Komal** defines the **operand representation**, converting registers and memory expressions into ModR/M, SIB, and displacement fields.

**Aditi** defines the **value and address-resolution stage**, handling immediates, displacements, relative jumps, labels, data directives, sections, Little-Endian representation, and the two-pass assembly process.

Together, the three components form the complete encoding pipeline:

```
Assembly Instruction
        ↓
Opcode Selection
        ↓
Operand Encoding
        ↓
Immediate / Displacement / Label Resolution
        ↓
Little-Endian Representation
        ↓
Final IA-32 Machine Code
```

The resulting specification provides a systematic method for parsing assembly instructions, resolving symbols and addresses, and generating variable-length IA-32 machine-code bytes.